ניתוח ביצועים:

גודל buffer	זמן
1	122.835 שניות
64	1.944 שניות
512	0.365 שניות
1024	0.235 שניות
8192	0.181 שניות
65536	0.158 שניות

ניתן לראות בבירור שבאפר גדול יותר מקטין משמעותית את זמן הריצה. הסיבה לכך לדעתי היא שbuffer גדול מקטין את מספר קריאות הsyscall. הפער המשמעותי ביותר הוא בין בית אחד ל64 כיוון שפרט לניצול תוכנתי טוב יותר והפחתת מס' הקריאות אנו מנצלים את החומרה טוב יותר היות ובמעבדים מודרניים גודל הרגיסטרים הוא 64 ביטים לכן במעבר בין בית בודד ל64 בתים אנו מגיעים לניצול מקסימלי של רוחב הפס של המעבד (היינו מגיעים לכך גם ב8 בתים כיוון שבית = 8 ביטים) בניגוד לקריאות הבאות שם אנו נמצאים כבר במצב האידיאלי מהבחינה הזאת והשיפור הוא תוכנתי בלבד.

O_EXCL:

*את התשובה מצאתי בג'מיני בעזרת הפרומט "תסביר את הדגל O_EXCL"

הדגל בשילוב הדגל O_CREAT דואג שבעת פתיחת קובץ, אם הקובץ לא קיים יווצר קובץ ריק חדש בשם המבוקש בעוד שאם קיים קובץ בשם זה הפקודה תכשל. על פי תקן posix שימוש בדגל ללא O_CREAT גורר התנהגות לא צפויה (שאלת המשך: "מה קורה בשימוש בליO_CREAT ").

Man 2 open נתן תשובה סך הכל זהה (כנראה שהשובה של ג'מיני התבססה עליו) והוסיף כי במקרים של symbolic link (מבציע לקובץ אחר) השימוש בדגלים O_CREAT | O_EXCL יגרום לפקודה להיכשל ללא קשר למה הקובץ מצביע עלי

*את התשובה מצאתי בעזרת הפרומט "מה ההבדל בין man open לman open 2":

ספריית הman של לינוקס מחולקת למדורים וללא מספר המדור הפקודה תחזיר את התוצאה הראשונה שהיא מוצאת לפי סדר המדורים.

Man open - פקודת הטרמינל open(נמצא בקטגוריה 1). פקודה זאת לא נמצאת בכל הגרסאות של לינוקס

Man 2 open – ה-syscall של הקרנל בו השתמשתי במטלת הקוד

AI Experiment

בחרתי להשתמש בGEMINI היות ויש לי מנוי חינמי.

כמי שמשתמש בAI ליצירת קוד בין אם לצרכים אישיים ובין אם בעבודה, אני יודע כמה קל לו לפספס המון דברים כאשר נותנים לו פרומטים קצרים ולא מפורטים מספיק. לכן ניסחתי את הפרומט הבא:

“Generate a code in C that receive 3 params: src file, dst file and buffer size and copy the content of src to dst in chunks with the size of buffer. the code must validate the ammount of args. also if src file exist exit with an error. it must use the syscalls read, write, open and close”

התוצאה הייתה מצוינת וסיפקה קוד שעשה את הנדרש. הקוד פעל בצורה דומה מאוד לשלי ולא נראה שונה במיוחד. ההבדלים היחידים היו בהודעות השגיאה שGEMINI ייצר יותר אינפורמטיביות (אני ניסיתי להיצמד לדוגמאות מההנחיות)